Section 6: Memory-Augmented and RAG Architectures

Refined Outline for Literature Review on LLM Context Management

J.C. Vaught

January 27, 2026

1 Introduction

Memory-augmented and Retrieval-Augmented Generation (RAG) architectures represent a paradigm shift in extending the effective context window of large language models beyond their architectural limitations. Rather than processing all information within the transformer's fixed context window, these approaches leverage external memory systems, databases, and retrieval mechanisms to dynamically incorporate relevant information during generation. This section examines the foundational architectures, retrieval strategies, advanced RAG variants, and agentic memory systems that enable models to access and utilize information far exceeding their native context capacity.

2 Foundational Memory Architectures

2.1 Neural Turing Machines

2.1.1 Architecture and Mechanism

- **Graves et al. (2014)** – "Neural Turing Machines" introduces differentiable external memory coupled to neural networks via attentional processes [1]
- Combines fuzzy pattern matching capabilities of neural networks with algorithmic power of programmable computers
- Differentiable end-to-end architecture enabling gradient descent training
- Analogous to Turing Machine or Von Neumann architecture

2.1.2 Capabilities and Results

- Demonstrates ability to infer simple algorithms: copying, sorting, associative recall
- Foundation for subsequent memory-augmented architectures
- Establishes attention-based memory read/write operations

2.2 Memory Networks

2.2.1 Original Memory Networks

- **Weston et al. (2015)** – Introduces memory networks with explicit storage and reasoning components
- Four-component architecture: input feature map, generalization, output feature map, response

2.2.2 End-to-End Memory Networks

- **Sukhbaatar et al. (2015)** – "End-To-End Memory Networks" enables fully differentiable training [2]
- Authors: Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, Rob Fergus (Facebook AI Research & NYU)
- Recurrent attention model over large external memory
- Multiple computational hops per output symbol
- Extension of attention mechanisms (RNNsearch) to multi-hop reasoning

2.2.3 Applications and Performance

- Synthetic question answering: competitive with supervised Memory Networks
- Language modeling: comparable to RNNs and LSTMs on Penn TreeBank and Text8
- Significantly reduced supervision requirements compared to original Memory Networks
- Published at NIPS 2015

3 Retrieval-Augmented Generation (RAG)

3.1 Foundational RAG Architecture

3.1.1 Original RAG Framework

- **Lewis et al. (2020)** – "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" [3]
- Full authors: Patrick Lewis et al. (12 authors total)
- Published at NeurIPS 2020; arXiv: 2005.11401
- Meta AI Research (formerly Facebook AI)

3.1.2 Core Components

- **Parametric memory**: Pre-trained seq2seq transformer model
- **Non-parametric memory**: Dense vector index of Wikipedia
- **Retrieval mechanism**: Dense passage retrieval for relevant context
- **Generation**: Conditions on retrieved passages for output generation

3.1.3 RAG Formulations

- **RAG-Sequence**: Conditions on same retrieved passages across entire generation
- **RAG-Token**: Can use different passages per token for dynamic retrieval

3.1.4 Performance and Impact

- State-of-the-art results on three open-domain QA tasks
- Generates more specific, diverse, and factual language than parametric-only baselines
- Foundation for modern RAG systems and retrieval-augmented architectures

3.2 Dense Passage Retrieval (DPR)

3.2.1 Motivation and Architecture

- **Karpukhin et al. (2020)** – "Dense Passage Retrieval for Open-Domain Question Answering" [4]
- Authors: Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih
- Published at EMNLP 2020; arXiv: 2004.04906
- Facebook AI Research

3.2.2 Technical Approach

- **Dual-encoder framework**: Separate BERT-based encoders for queries and passages
- Question Encoder: Processes input query into dense representation
- Passage Encoder: Encodes documents/passages into dense vectors
- Training: Simple dual-encoder framework with small number of question-passage pairs
- Retrieval: Dense representations alone (no sparse features)

3.2.3 Performance Improvements

- 9-19% absolute improvement in top-20 passage retrieval accuracy over Lucene-BM25
- State-of-the-art results on multiple open-domain QA benchmarks
- Reduces semantic search time from 65 hours (BERT) to 5 seconds
- Widely adopted as foundation for RAG retrieval components

3.3 Retrieval-Enhanced Transformers (RETRO)

3.3.1 DeepMind's RETRO Architecture

- **Borgeaud et al. (2022)** – "Improving Language Models by Retrieving from Trillions of Tokens" [5]
- Authors: Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. (DeepMind)
- Published at ICML 2022; arXiv: 2112.04426

3.3.2 Key Innovation

- Retrieval from 2 trillion token database during pre-training and inference
- Achieves GPT-3 / Jurassic-1 performance with 25× fewer parameters
- Performance gain comparable to 10× parametric model size increase

3.3.3 Technical Architecture

- **Frozen BERT retriever**: Retrieves relevant passages from massive database
- **Differentiable encoder**: Processes retrieved information
- **Chunked cross-attention**: Integrates retrieved neighbors at passage level
- Interleaves regular self-attention (document level) with cross-attention (passage level)

3.3.4 Performance Results

- 7.5B parameter RETRO outperforms 175B Jurassic-1 on 10/16 datasets
- Outperforms 280B Gopher on 9/16 datasets
- RETROfitting: Pre-trained transformers can be rapidly adapted with retrieval

3.3.5 Database and Scalability

- Training database: Web pages, books, news articles, code
- 2 trillion tokens – order of magnitude more data than typical training
- Demonstrates scalability of retrieval-augmented approaches

4 kNN-Augmented Memory Mechanisms

4.1 Memorizing Transformers

4.1.1 Architecture and Approach

- **Wu et al. (2022)** – "Memorizing Transformers" [6]
- Authors: Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, Christian Szegedy
- Published at ICLR 2022 (Spotlight); arXiv: 2203.08913

4.1.2 Core Mechanism

- **Approximate kNN lookup**: Non-differentiable memory of recent (key, value) pairs
- Enables reading and memorizing new data at inference time
- Immediate knowledge acquisition without retraining
- Memory size scales up to 262K tokens

4.1.3 Performance Across Domains

- **Generic web text**: C4 dataset improvements
- **Math papers**: arXiv dataset gains
- **Books**: PG-19 long-form content
- **Code**: GitHub repositories
- **Formal theorems**: Isabelle proof assistant
- Capable of utilizing newly defined functions and theorems during test time

4.1.4 Scaling Behavior

- Performance steadily improves with memory size increase
- Effective for code and mathematical reasoning with dynamic knowledge
- Demonstrates test-time adaptation capabilities

4.2 ColBERT: Late Interaction Retrieval

4.2.1 Late Interaction Architecture

- **Khattab & Zaharia (2020)** – "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT" [7]
- Authors: Omar Khattab, Matei Zaharia (Stanford)
- Published at SIGIR 2020; arXiv: 2004.12832

4.2.2 Technical Innovation

- **Independent encoding**: Query and document encoded separately with BERT
- **Late interaction**: Cheap yet powerful interaction step models fine-grained similarity
- **MaxSim operator**: Maximum cosine similarity of each query vector with document vectors
- Combines expressiveness of deep LMs with offline document pre-computation

4.2.3 Efficiency and Performance

- Two orders of magnitude faster than BERT-based models
- Four orders of magnitude fewer FLOPs per query
- Competitive effectiveness with existing BERT models
- Outperforms all non-BERT baselines
- Enables end-to-end retrieval from large document collections via vector-similarity indexes

4.2.4 Extensions: ColBERTv2

- **Santhanam et al. (2022)** – "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction"
- Published at NAACL 2022; arXiv: 2112.01488
- Further efficiency improvements and optimizations

5 Advanced RAG Variants

5.1 Self-RAG: Self-Reflective Retrieval

5.1.1 Framework Overview

- **Asai et al. (2023)** – "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection" [8]
- Authors: Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, Hannaneh Hajishirzi
- Published at ICLR 2024 (Oral – top 1%); arXiv: 2310.11511

5.1.2 Key Innovation: Reflection Tokens

- **Adaptive retrieval**: On-demand retrieval decisions
- **Reflection tokens**: Special tokens for self-critique and evaluation
- Model can retrieve multiple times or skip retrieval entirely
- Generates and reflects on retrieved passages and own generations

5.1.3 Advantages Over Traditional RAG

- Traditional RAG: Indiscriminate retrieval of fixed number of passages
- Self-RAG: Adaptive decisions on when and what to retrieve
- Self-critique mechanisms improve relevance filtering
- Enhanced LM versatility and response quality

5.1.4 Performance Results

- Self-RAG (7B, 13B) outperforms ChatGPT on open-domain QA
- Superior performance on reasoning and fact verification tasks
- Significant gains in factuality and citation accuracy for long-form generation
- State-of-the-art among LLMs and retrieval-augmented models

5.2 Corrective RAG (CRAG)

5.2.1 Framework and Approach

- **Yan et al. (2024)** – "Corrective Retrieval Augmented Generation" [9]
- arXiv: 2401.15884 (submitted January 2024, revised October 2024)

5.2.2 Core Mechanism: Retrieval Evaluator

- **Lightweight retrieval evaluator**: Assesses quality of retrieved documents
- **Confidence degree**: Triggers different knowledge retrieval actions
- Self-assessment before generation improves accuracy and relevance

5.2.3 Three Action Types

- **Correct**: Reliable retrieval
  - Decompose-then-recompose technique
  - Remove irrelevant information
  - Retain key knowledge
- **Incorrect**: Unreliable retrieval
  - Discard faulty retrievals
  - Trigger web search for reliable information
- **Ambiguous**: Unclear confidence
  - Combine refined retrieval with web search
  - Ensure diverse and robust knowledge integration

5.2.4 Integration with Other RAG Methods

- **Self-CRAG**: Inserts CRAG into Self-RAG framework
- Replaces retrieved items with processed knowledge (Correct), external knowledge (Incorrect), or combined knowledge (Ambiguous)
- Demonstrates adaptability across RAG-based approaches

5.2.5 Performance

- Tested on four datasets: short-form and long-form generation
- Significantly outperforms traditional RAG and Self-RAG
- Improved performance across various benchmarks

5.3 Graph RAG

5.3.1 Microsoft GraphRAG Framework

- **Microsoft Research** – GraphRAG: Structured, hierarchical RAG approach
- Available at: `github.com/microsoft/graphrag`

5.3.2 Core Approach

- **Knowledge graph extraction**: LLM-generated graphs from raw text
- **Community hierarchy**: Structured organization of entities and relationships
- **Community summaries**: Hierarchical abstractions
- **Graph machine learning**: Augments prompts at query time

5.3.3 Advantages Over Baseline RAG

- **Connecting disparate information**:
  - Baseline RAG struggles with aggregation queries (e.g., "What are the top 5 themes?")
  - Vector search limited to semantically similar text snippets
  - Graph RAG connects information via relationships
- **Better context and hidden connections**:
  - Captures relationships beyond semantic similarity
  - Uncovers hidden but crucial correlations
  - More accurate and relevant results

5.3.4 Technical Components

- Text extraction from documents
- Network analysis for relationship modeling
- LLM-based entity and relation extraction
- Community detection and hierarchical clustering
- Query-time graph traversal and augmentation

5.3.5 Applications and Availability

- Complex queries requiring relationship understanding
- Large-scale document analysis
- Scientific research: Microsoft Discovery platform in Azure
- LazyGraphRAG variant for efficiency

6 Agentic Memory Systems

6.1 MemGPT: Virtual Context Management

6.1.1 Operating System Analogy

- **Packer et al. (2023)** – "MemGPT: Towards LLMs as Operating Systems" [10]
- Authors: Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, Joseph E. Gonzalez
- arXiv: 2310.08560 (October 2023)

6.1.2 Core Concept: Hierarchical Memory

- **Virtual context management**: Inspiration from OS hierarchical memory systems
- Provides appearance of large memory via data movement between fast and slow memory
- Fixed-context LLM processor augmented with hierarchical memory system

6.1.3 Memory Architecture

- **Main context** (prompt tokens):
  - System instructions
  - Working context
  - FIFO queue for recent interactions
- **External context**:
  - Archival storage database
  - Recall storage database
- **Functions**: LLM manages own memory via function calls
- **Interrupts**: Manages control flow between system and user

6.1.4 Applications and Evaluation

- **Document analysis**:
  - Analyze documents far exceeding LLM context window
  - Dynamic loading of relevant document sections
- **Multi-session chat**:
  - Conversational agents that remember across sessions
  - Reflection and evolution through long-term interactions
  - Persistent memory across conversation boundaries

6.1.5 Implementation and Availability

- Open-source implementation: `memgpt.ai`
- Rebranded as "Letta" for production deployment
- Demonstrates unbounded context through memory hierarchy

7 Retrieval Infrastructure and Optimization

7.1 Embedding Models

7.1.1 Sentence-BERT (SBERT)

- **Reimers & Gurevych (2019)** – "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" [11]
- Authors: Nils Reimers, Iryna Gurevych
- Published at EMNLP-IJCNLP 2019; arXiv: 1908.10084

7.1.2 Technical Approach

- **Siamese and triplet network structures**: Derive semantically meaningful embeddings
- **Pooling operation**: Fixed-size sentence embeddings from BERT output
- **Dual-encoder fine-tuning**: Simultaneous processing of two inputs
- Embeddings comparable via cosine similarity

7.1.3 Efficiency Gains

- BERT: 50M inference computations (65 hours) for 10K sentence similarity
- SBERT: 5 seconds for same task
- Maintains BERT-level accuracy with massive speedup

7.1.4 Applications

- Semantic search and information retrieval
- Semantic textual similarity
- Paraphrase mining
- Foundation for RAG embedding components
- Official implementation: `sbert.net`

7.1.5 Modern Embedding Models

- `multi-qa-mpnet-base-dot-v1`: Optimized for question-answering
- `all-MiniLM-L6-v2`: Lightweight general-purpose embeddings
- OpenAI `text-embedding-ada-002`: Commercial state-of-the-art
- Cohere Embed v3: Multilingual and multi-domain
- Context window constraints: typically 512-8192 tokens

7.2 Chunking Strategies for RAG

7.2.1 Fixed-Size Chunking

- Segments text into equal-sized pieces by character, token, or word count
- **Recommended starting point**: 250 tokens (1000 characters)
- **Overlap**: Maintains continuity across chunk boundaries
- Simple, predictable, widely applicable

7.2.2 Recursive Character Splitting

- **Default choice**: 80% of RAG applications
- Balances simplicity with structure awareness
- Process:
  1. Chunk by inherent separators (paragraphs, sections)
  2. Split further if chunk exceeds size limit
- Respects document structure while enforcing size constraints

7.2.3 Semantic Chunking

- Groups sentences by semantic similarity of embeddings
- Analyzes relatedness of consecutive sentences
- Creates chunks where topics shift
- More expensive but preserves semantic coherence

7.2.4 Page-Level Chunking

- **NVIDIA 2024 benchmark**: Tested 7 strategies across 5 datasets
- Page-level chunking: 0.648 accuracy, lowest std dev (0.107)
- Best for structured documents with clear page boundaries

7.2.5 Late Chunking

- Feed entire document into long-context embedding model
- Create token-level embeddings with full context understanding
- Split into chunks after embedding
- Preserves global context in embeddings

7.2.6 Hierarchical Chunking

- Multiple layers at different granularities
- Top layer: Broad sections/themes
- Lower layers: Fine-grained details
- Enables multi-resolution retrieval

7.2.7 LLM-Based / Agentic Chunking

- LLM determines splitting based on semantic meaning
- Considers paragraph types, section headings, content structure
- Experimental but promising for complex documents

7.2.8 Query-Type Optimization

- **Factoid queries**: 256-512 tokens optimal
- **Analytical queries**: 1024+ tokens needed
- Chunk size should match query complexity

7.2.9 Best Practices

- Chunking is arguably most important factor for RAG performance
- Test multiple strategies for specific use case
- Consider embedding model context window constraints
- Balance chunk size with retrieval precision/recall

8 Future Directions and Open Challenges

8.1 Scalability

- Retrieval from trillion-token+ databases
- Efficient indexing and search at massive scale
- Dynamic knowledge base updates without retraining

8.2 Integration and Unification

- Combining parametric and non-parametric memory
- Unified training objectives for retrieval and generation
- End-to-end differentiable retrieval mechanisms

8.3 Reliability and Factuality

- Improved retrieval relevance assessment
- Handling conflicting information from multiple sources
- Citation accuracy and provenance tracking
- Mitigating retrieval-induced hallucinations

8.4 Efficiency

- Reducing latency of retrieval-augmented inference
- Compression of retrieved context
- Adaptive retrieval frequency and granularity
- On-device RAG for privacy-sensitive applications

8.5 Multimodal Memory

- Retrieval across text, images, audio, video
- Cross-modal memory architectures
- Unified representations for multimodal retrieval

9 Conclusion

Memory-augmented and RAG architectures represent a fundamental shift from purely parametric language models to hybrid systems that leverage external knowledge. From foundational work on Neural Turing Machines and Memory Networks to modern systems like RETRO, Self-RAG, and MemGPT, these approaches demonstrate that effective context extension requires not just larger windows but intelligent retrieval, organization, and integration of external information. The progression from basic retrieval to self-reflective, corrective, and graph-based methods shows increasing sophistication in when, what, and how to retrieve. As language models continue to scale, memory-augmented architectures will remain essential for grounding generation in factual knowledge, enabling long-term memory, and providing systems with access to information beyond their training cutoff.

Key References

References

[1] Graves, A., Wayne, G., & Danihelka, I. (2014). *Neural Turing Machines*. arXiv preprint arXiv:1410.5401.

[2] Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2015). *End-To-End Memory Networks*. In Advances in Neural Information Processing Systems (NeurIPS) 2015.

[3] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems (NeurIPS) 2020. arXiv:2005.11401.

[4] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. In Proceedings of EMNLP 2020, pp. 6769–6781. arXiv:2004.04906.

[5] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). *Improving Language Models by Retrieving from Trillions of Tokens*. In Proceedings of the 39th International Conference on Machine Learning (ICML) 2022. arXiv:2112.04426.

[6] Wu, Y., Rabe, M. N., Hutchins, D., & Szegedy, C. (2022). *Memorizing Transformers*. In International Conference on Learning Representations (ICLR) 2022 Spotlight. arXiv:2203.08913.

[7] Khattab, O., & Zaharia, M. (2020). *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 39–48. arXiv:2004.12832.

[8] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. In International Conference on Learning Representations (ICLR) 2024 Oral (top 1%). arXiv:2310.11511.

[9] Yan, S., et al. (2024). *Corrective Retrieval Augmented Generation*. arXiv preprint arXiv:2401.15884.

[10] Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv preprint arXiv:2310.08560.

[11] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992. arXiv:1908.10084.